第9小时

系统架构设计基础知识

9.0 章节考点分析

第 9 小时主要学习软件架构的基本概念、基于架构的软件开发方法、软件架构风格、软件架构复用以及特定领域软件体系结构等内容。本小时内容侧重于概念知识，考查的知识点来源于教材。根据考试大纲，本小时知识点会涉及单项选择题（约占 8～15 分）和下午案例题（25 分），论文也会有涉及。本小时知识架构如图 9.1 所示。

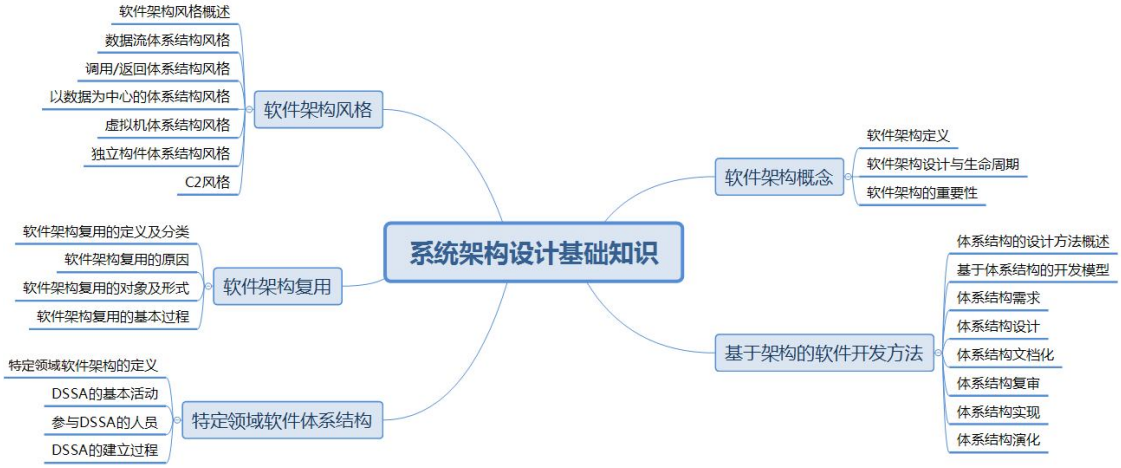

图 9.1　本小时知识架构

【导读小贴士】

软件架构设计是降低成本、改进质量、按时和按需交付产品的关键因素。体系架构的方法和步骤作为系统架构设计师考试最基本、最核心的方法论，需要重点理解并掌握。很多案例分析题也都来自本小时内容。

9.1 软件架构概念

【基础知识点】

1. 软件架构定义

软件架构（Software Architecture）或称软件体系结构，是指系统的一个或者多个结构，这些结构包括软件的构件（可能是程序模块、类或者是中间件）、构件的外部可见属性及其之间的相互关系。体系结构的设计包括数据库设计和软件结构设计，后者主要关注软件构件的结构、属性和交互作用，并通过多种视图全面描述。

2. 软件架构设计与生命周期

软件架构是贯穿整个生命周期的，不同阶段的作用和意义不同，各阶段架构工作见表 9.1。

表 9.1 各阶段架构工作一览表

阶段	作用和意义
需求分析阶段	有利于各阶段参与者的交流，也易于维护各阶段的可追踪性
设计阶段	关注的最早和最多的阶段
实现阶段	有效实现从软件架构设计向实现的转换
构件组装阶段	可复用构件组装的设计能够提高系统实现的效率
部署阶段	组织和展示部署阶段的软硬件架构、评估分析部署方案
后开发阶段	主要围绕维护、演化、复用进行

（1）需求分析阶段。需求分析阶段软件架构研究还处于起步阶段。需求关注的是问题空间，架构关注的是解空间，需要保持二者的可追踪性和转换。从软件需求模型向软件架构模型的转换主要关注两个问题：

1）如何根据需求模型构建软件架构模型。

2）如何保证模型转换的可追踪性。

（2）设计阶段。这一阶段的研究主要包括：软件架构模型的描述、软件架构模型的设计与分析方法，以及对软件架构设计经验的总结与复用等。其中架构模型的描述研究包括：

1）组成 SA 模型（软件架构模型）的基本概念。即构件和连接子的建模。

2）体系架构描述语言（Architecture Describe Language，ADL）。是用于描述软件体系架构的语言，与其他建模语言最大的区别在于其更关注构件间互联机制（连接子），典型的 ADL 语言包括 Unicon、Rapide、Darwin、Wright、C2SADL、Acme、XADLOL、XYZ/ADL 和 ABC/ADL 等。

3）多视图。反映的是一组系统的不同方面，体现了关注点分散的思想，通常与 ADL 结合起来描述系统的体系结构。典型的模型包括：4+1 模型、Hofmesiter 的 4 视图模型、CMU-Sei 的 Views and Beyond 模型。视图标准包括：IEEE 的 I471-2000、RM-ODP、UML 以及 IBM 的 Zachman。

（3）实现阶段。这一阶段的体系结构研究的内容有：

1）基于 SA 的开发过程支持。

2）寻求从 SA 向实现过渡的途径。

3）研究基于 SA 的测试技术。

缩小软件架构设计与底层实现概念差距的手段：模型转换技术、封装底层的实现细节、在 SA 模型中引入实现阶段的概念（如用程序设计语言描述）。

（4）构件组装阶段。研究的内容包括：

1）如何支持可复用构件的互联，即对 SA 设计模型中规约的连接子的实现提供支持。

2）组装过程中，如何检测并消除体系结构失配问题。这些问题主要包括：①构件本身的失配；②连接子（互联机制）的失配；③部分和整体的失配。

（5）部署阶段。部署阶段的软件架构对软件部署的作用：一是提供高层的体系结构视图描述部署阶段的软硬件模型；二是基于软件架构模型可以分析部署方案的质量属性，从而选择合理的部署方案。

（6）后开发阶段。部署安装后（后开发阶段）的系统架构研究方向包括：动态软件体系结构、体系结构恢复与重建。体系结构重建的方法有：手工体系结构重建、工具支持的手工重建、通过查询语言来自动建立聚集、使用其他技术（如数据挖掘）。

3. 软件架构的重要性

软件架构设计是降低成本、改进质量、按时和按需交付产品的关键因素。软件架构的重要性包括：

（1）架构设计能够满足系统的品质。

（2）架构设计使受益人达成一致的目标。

（3）架构设计能够支持计划编制过程。

（4）架构设计对系统开发的指导性。

（5）架构设计能够有效地管理复杂性。

（6）架构设计为复用奠定了基础。

（7）架构设计能够降低维护费用。

（8）架构设计能够支持冲突分析。

9.2 基于架构的软件开发方法

【基础知识点】

1. 体系结构的设计方法概述

基于体系结构（架构）的软件设计（Architecture-Based Software Design，ABSD）方法是体系结构驱动的，即指构成体系结构的商业、质量和功能需求的组合驱动的。在基于体系结构的软件设计方法中，采用视角与视图来描述软件架构，采用用例来描述功能需求，采用质量场景来描述质量需求。ABSD 方法具有三个基础：功能的分解、通过选择体系结构风格来实现质量和商业需求、软件模板的使用。ABSD 是自顶向下、递归细化的，迭代的每一步都有清晰的定义，有助于降低体系结构设计的随意性。

2. 基于体系结构的开发模型

传统的软件开发模型开发效率较低，ABSDM 模型把整个基于体系结构的软件开发过程划分为体系结构需求、设计、文档化、复审、实现和演化六个子过程，如图 9.2 所示。

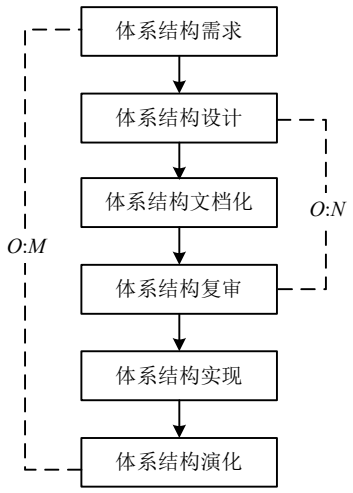

图 9.2　基于体系结构的软件开发过程

3. 体系结构需求

体系结构的需求工作包括获取用户需求和标识系统中拟用构件。

（1）需求获取。体系结构需求的获取一般来自三个方面：质量目标、系统的商业目标和系统开发人员的商业目标。

（2）标识构件。标识构件分三步完成：生成类图→对类进行分组→把类打包成构件。

（3）架构需求评审的审查重点包括需求是否真实反映了用户的要求、类的分组是否合理、构

件合并是否合理等。

4. 体系结构设计

软件的体系设计过程：提出软件体系结构模型→映射构件→分析构件相互作用→产生体系结构设计评审。设计评审必须邀请独立于系统开发的外部人员。

5. 体系结构文档化

体系结构文档化过程的主要输出结果是体系结构规格说明和测试体系结构需求的质量设计说明书。

6. 体系结构复审

一个主版本的软件体系结构分析之后，要安排一次由外部人员（用户代表和领域专家）参加的复审。复审的目的是标识潜在的风险，及早发现体系结构设计中的缺陷和错误，必要时，可搭建一个可运行的最小化系统用于评估和测试体系结构是否满足需要。

7. 体系结构实现

体系结构的实现过程是以复审后的文档化体系结构说明书为基础的，具体为：分析与设计→构件实现→构件组装→系统测试。体系结构说明书中定义了系统中构件与构件之间的关系。测试包括单个构件的功能性测试及被组装应用的整体功能和性能测试。

8. 体系结构演化

体系结构演化史使用系统演化步骤去修改应用，以满足新的需求。系统演化步骤为：需求变化归类→体系结构演化计划→构件变动→更新构件的相互作用→构件组装与测试→技术评审→演化后的体系结构。

9.3 软件架构风格

【基础知识点】

1. 软件架构风格概述

软件体系结构设计的核心目标是重复的体系结构模式（软件复用/重用）。软件体系结构（架构）风格是描述某一特定应用领域中系统组织方式的惯用模式。体系结构风格定义一个系统家族，即一个体系结构定义一个词汇表和一组约束。

（1）词汇表：包含构件和连接件。

（2）约束：约束定义构件和连接件的组合方式。

体系结构风格反映了领域中众多系统所共有的结构和语义特性，并指导如何将各个模块和子系统有效地组织成一个完整的系统。

2. 数据流体系结构风格

（1）批处理体系结构风格：每个处理步骤是一个独立的程序，每一步必须在前一步结束后才能开始，且数据必须是完整，以整体的方式传递。

（2）管道和过滤器：把系统分为几个序贯地处理步骤，每个步骤之间通过数据流连接，一个

步骤的输出是另一个步骤的输入，每个处理步骤都有输入和输出，如图 9.3 所示。

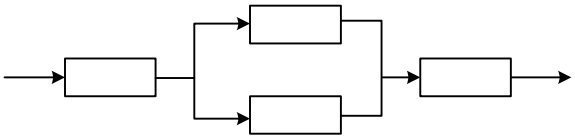

图 9.3　管道和过滤器风格

3. 调用/返回体系结构风格

调用-返回风格在系统中采用了调用与返回机制。利用调用-返回实际上是一种分而治之的策略，主要思想是将一个复杂的大系统分解为若干个子系统，降低复杂度，增加可修改性。

（1）主程序/子程序风格：采用单线程控制，把问题划分为若干处理步骤，构件即为主程序和子程序。

（2）面向对象体系结构风格：构件是对象，即抽象数据类型的实例，如图 9.4 所示。

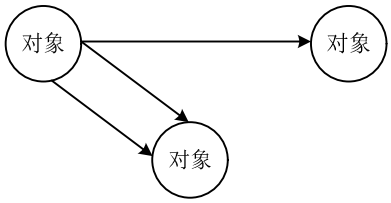

图 9.4　数据抽象和面向对象组

（3）层次型体系结构风格：每一层为上层服务，并作为下层的接口。仅相邻层间具有层接口，如图 9.5 所示。

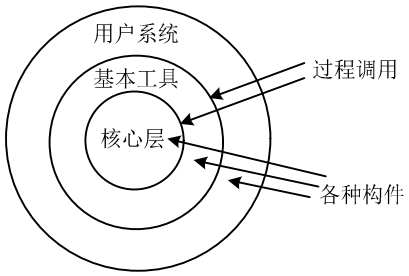

图 9.5　层次型体系结构风格

（4）客户端/服务器体系结构风格。

1）二层 C/S 模式。主要组成部分：数据库服务器（后台：负责数据管理）、客户应用程序（前台：完成与用户交互任务）和网络。

优点：客户应用和服务器构件分别运行在不同的计算机上。

缺点：开发成本高，客户端设计复杂，信息内容和形式单一，不利于推广，软件移植困难，软件维护和升级困难。

2）三层 C/S 模式：瘦客户端模式。应用该功能分为表示层、功能层和数据层。

表示层：用户接口与应用逻辑层的交互，不影响业务逻辑，通常使用图形用户界面。

功能层：实现具体的业务处理逻辑。

数据层：数据库管理系统。

（5）浏览器/服务器风格（B/S）。

1）B/S 风格：是三层应用结构的实现方式，其三层结构分别为：浏览器；Web 服务器；数据库服务器。

2）相比于 C/S 的不足之处：动态页面的支持能力弱、系统拓展能力差、安全性难以控制、响应速度不足、数据交互性不强。

4. 以数据为中心的体系结构风格

（1）仓库体系结构风格：存储和维护数据的中心场所。由中央数据结构（说明当前数据状态）和一组独立构件（对中央数据进行操作）组成，如图 9.6 所示。

（2）黑板体系结构风格：是一种问题求解模型，是组织推理步骤、控制状态数据和问题求解之领域知识的概念框架。可通过选取各种黑板、知识源和控制模块的构件来设计，应用于信号处理领域，如语音识别和模式识别，如图 9.7 所示。

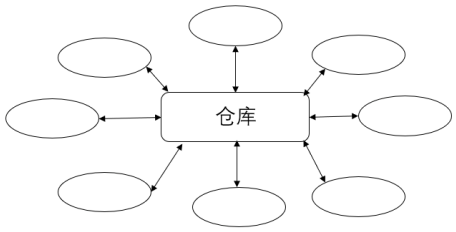

图 9.6　仓库体系结构风格

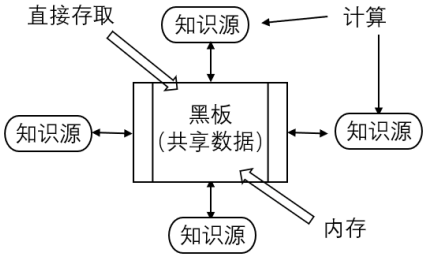

图 9.7　黑板体系结构风格

5. 虚拟机体系结构风格

虚拟机体系结构风格基本思想是人为构建一个运行环境，可以解析与运行自定义的一些语言，增加架构的灵活性。

（1）解释器体系结构风格：通常被用来建立一种虚拟机以弥合程序语义与硬件语义之间的差异，缺点是执行效率较低，典型例子是专家系统，如图 9.8 所示。

（2）规则系统体系结构风格：包括知识库、规则解释器、规则/数据选择器及工作内存（程序运行存储区），如图 9.9 所示。

6. 独立构件体系结构风格

独立构件体系结构风格强调系统中的每个构件都是相对独立的个体，它们之间不直接通信，以

降低耦合度，提升灵活度。

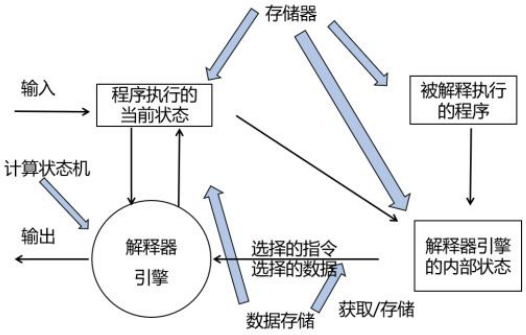

图 9.8　解释器体系结构风格

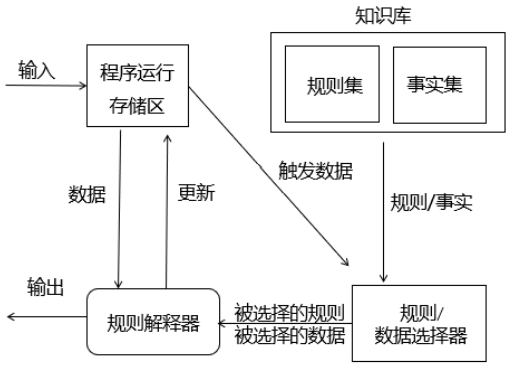

图 9.9　规则系统体系结构风格

（1）进程通信体系结构风格：构件是独立的过程，连接件是消息传递。

（2）事件系统体系结构风格：构件不直接调用一个过程，而是触发或广播一个或多个事件。

7. C2 风格

C2 风格通过连接件连接构件或某个构件组，构件与构件之间无连接，如图 9.10 所示。

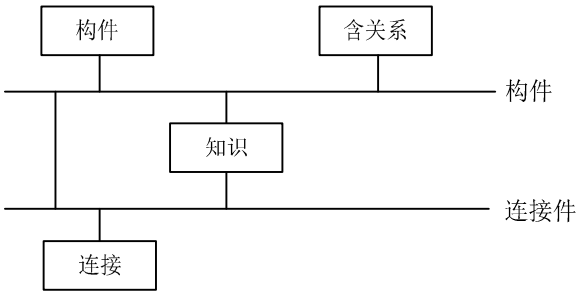

图 9.10　C2 风格

9.4 软件架构复用

【基础知识点】

（1）软件架构复用的定义及分类。软件复用是系统化的软件开发过程：开发一组基本的软件构件模块，以覆盖不同的需求/体系结构之间的相似性，提高系统开发的效率、质量和性能。

软件架构复用的类型包括机会复用和系统复用。机会复用是在开发过程中，只要发现有可复用的资产就复用。系统复用是在开发前进行规划，决定哪些复用。

（2）软件架构复用的原因：减少开发工作、减少开发事件、降低开发成本、提高生产力、提高产品质量，更好的互操作性。

（3）软件架构复用的对象及形式。可复用的资产包括：需求、架构设计、元素、建模分析、测试、项目规划、过程+方法+工具、人员、样本系统、缺陷消除。一般形式的复用包括：函数的复用、库的复用、面向对象开发中的类、接口和包的复用。

（4）软件架构复用的基本过程。首先构建/获取可复用的软件资产（复用前提）→管理可复用资产→使用可复用资产。

9.5 特定领域软件体系结构

【基础知识点】

1. 特定领域软件架构的定义

特定领域软件架构（Domain Specific Software Architecture，DSSA）是在一个特定应用领域中为一组应用提供组织结构参考的标准软件体系结构，即用于某一类特定应用领域的标准软件构件集合。DSSA 的特征：领域性、普遍性、抽象性、可复用性。

2. DSSA 的基本活动

DSSA 的基本活动有领域分析、领域设计、领域实现。

（1）领域分析：通过分析领域中系统的共性需求，建立领域模型。

（2）领域设计：设计 DSSA，且 DSSA 需要具备领域需求变化的适应性。

（3）领域实现：获取可重用信息。

3. 参与 DSSA 的人员

参与 DSSA 的人员有领域专家、领域分析师、领域设计人员和领域实现人员。

4. DSSA 的建立过程

DSSA 的建立过程是并发的、递归的、反复的螺旋模型，分为五个阶段：

（1）定义领域范围。

（2）定义领域特定元素。

（3）定义领域特定的设计和实现约束。

（4）定义领域模型和体系结构。

（5）产生、搜集可重用的单元。

9.6 练习题

1．软件架构风格是描述某一特定应用领域中系统组织方式的惯用模式。架构风格定义了一类架构所共有的特征，主要包括架构定义、架构词汇表和架构（　　）。

A．描述　　B．组织　　C．约束　　D．接口

解析：本题主要考查软件架构风格的定义。软件架构风格是描述某一特定应用领域中系统组织方式的惯用模式。架构风格定义了一类架构所共有的特征，主要包括架构定义、架构词汇表和架构约束。

答案：C

2．以下叙述中，（　　）不是软件架构的主要作用。

A．在设计变更相对容易的阶段，考虑系统结构的可选方案

B．便于技术人员与非技术人员就软件设计进行交互

C．展现软件的结构、属性与内部交互关系

D．表达系统是否满足用户的功能性需求

解析：本题主要考查软件架构基础知识。软件架构能够在设计变更相对容易的阶段，考虑系统结构的可选方案，便于技术人员与非技术人员就软件设计进行交互，能够展现软件的结构、属性与内部交互关系。但是软件架构与用户对系统的功能性需求没有直接的对应关系。

答案：D

3．特定领域软件架构（DSSA）是在一个特定应用领域中，为一组应用提供组织结构参考的标准软件体系结构。DSSA 通常是一个具有三个层次的系统模型，包括__（1）__环境、领域特定应用开发环境和应用执行环境，其中__（2）__主要在领域特定应用开发环境中工作。

（1）A．领域需求　　B．领域开发　　C．领域执行　　D．领域应用

（2）A．操作员　　B．领域架构师　　C．应用工程师　　D．程序员

解析：本题主要考查特定领域软件架构的基础知识。特定领域软件架构是在一个特定应用领域中，为一组应用提供组织结构参考的标准软件体系结构。DSSA 通常是一个具有三个层次的系统模型，包括领域开发环境、领域特定应用开发环境和应用执行环境，其中应用工程师主要在领域特定应用开发环境中工作。

答案：B　C